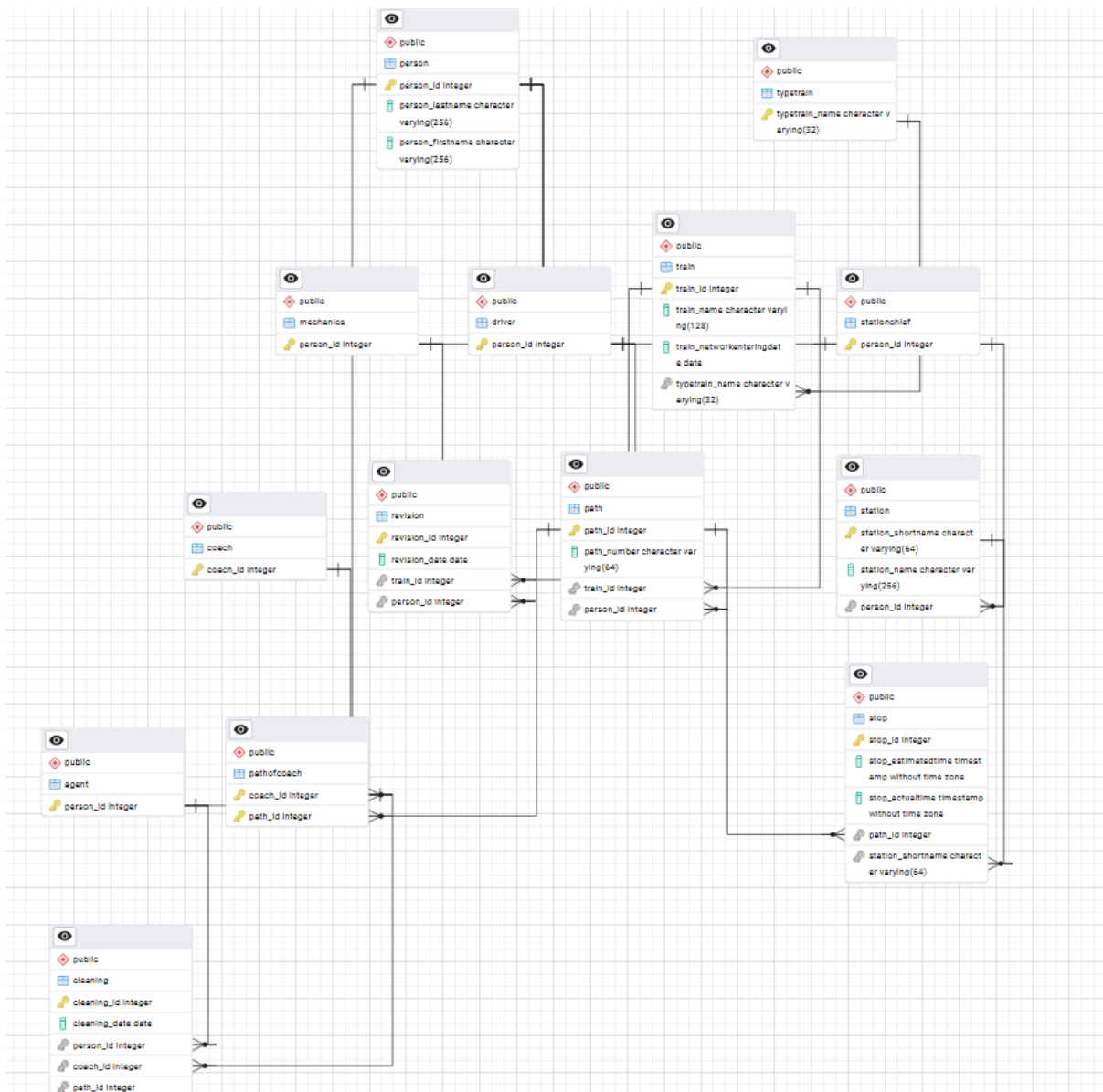


Report Lab 2 : Train DB

This report, done by Adriel and Duc, will show you through each question how we make the query, the output of the query, and, most importantly, how we did the query and what our thinking was.

First, we looked at the ERD diagram to have a better comprehension of how the database is constructed and how the tables are linked. We can clearly see all the columns of a table, what the type of data is, and all the relations. You will see that this is the most important phase because without it, we won't be able to make the queries with the SQL function **INNER JOIN**. To explain briefly what this function is in SQL, it allows you to glue two tables (or more) that are linked with a PK (Primary Key)/FK(Foreign Key).



1. Connect to the database. Have a look to the appendix for connection tutorial

As in the previous Lab, we need to pass through the query tool that allows us to be connected to the database and request the information we want. Here's how it looks:



Welcome to the Query Tool Workspace!

The Query Tool is a robust and versatile environment designed for executing SQL commands and reviewing result sets efficiently.

In this workspace, you can seamlessly open and manage multiple query tabs, making it easier to organize your work. You can select the existing servers or create a completely new ad-hoc connection to any database server as needed.

Let's connect to the server

Existing Server
(Optional)

trains_en

Server Name

trains_en

Host
name/address

appli-pfe.ec-nantes.fr

Port

5432

Database

postgres

User

ft_01

Password

Role

Select an item...

Service

Connection Parameters			+
	Name	Keyword	Value
🗑	SSL mode	sslmode	prefer
🗑	Connection timeout	connect_timeout	10

↺ Reset

🔌 Connect & Open Query Tool

2. Check the database is a right translation for the schema. You should have a look at the table names and the column names.

As we said, thanks to the ERD we have the entire diagram on the SQL DB, and now we can use the schemas (you gave) and ERD to see any difference; here are the differences we saw :

Schema	pgAdmin	Reason for the Difference
path	path	good (No difference)
cleaning	cleaning	good (No difference)
staff	agent	wrong table name (Different table name)
coachofpath	pathofcoach	wrong name (Different table name)
coach	coach	good (No difference)
mechanics	mechanics	good (No difference)
driver	driver	good (No difference)
typetrain	typetrain	good (No difference)
train	train	good (No difference)
trainstop	stop	The columns are not good (trainstop_estimatedtime != stop_estimatedtime ; trainstop_actualetime != stop_actualetime ; trainstop_id != stop_id)
station	station	Station but one column not good (station_longname != station_name)
revision	revision	good (No difference)
person	person	good (No difference)
stationchief	stationchief	good (No difference)

3.1. List the trains in the database :

We took the information from the train table and asked for all the trains in the DB. (We will show only the 10 first rows)

The screenshot shows a database query interface. At the top, there are tabs for 'Query' and 'Query History'. Below them is a text area containing the SQL query: `select typetrain_name from train`. Underneath the query area are tabs for 'Data Output', 'Messages', and 'Notifications'. A toolbar with various icons is visible, including a 'SQL' button. Below the toolbar, it says 'Showing rows: 1 to 597' and 'Page No: 1 of 1'. The main area displays a table with the following data:

	typetrain_name character varying (32)
1	TER
2	TGV
3	CORAIL
4	CORAIL
5	TER
6	TER
7	TER
8	TER
9	TER
10	CORAIL

3.2. List stations in the database using alphabetic ordering.

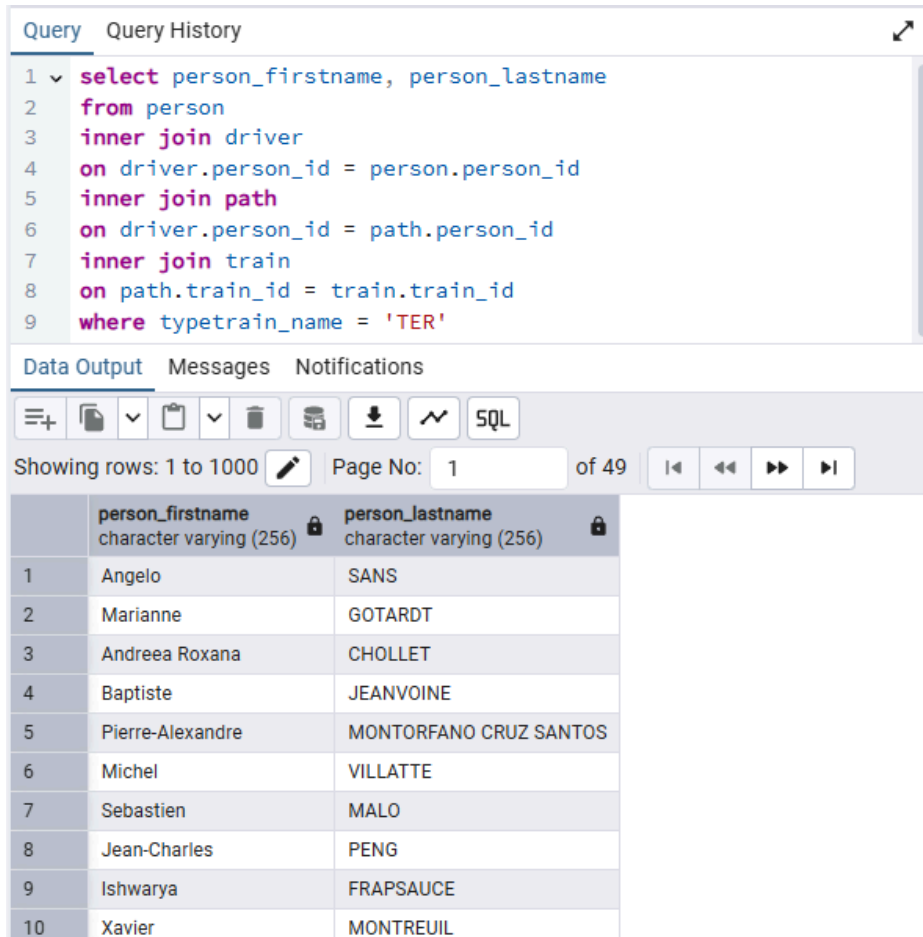
We select the `station_name` column from the `station` table and order it by the `station_name`. In SQL, by default, it's already in ascending order (ASC).

The screenshot shows a database query interface. At the top, there are tabs for 'Query' and 'Query History'. Below them is a text area containing the SQL query: `select station_name from station order by station_name`. Underneath the query area are tabs for 'Data Output', 'Messages', and 'Notifications'. A toolbar with various icons is visible, including a 'SQL' button. Below the toolbar, it says 'Showing rows: 1 to 1000' and 'Page No: 1 of 2'. The main area displays a table with the following data:

	station_name character varying (256)
1	Gare d'Abscon
2	Gare d'Accolay
3	Gare d'Achiet
4	Gare d'Agde
5	Gare d'Agen
6	Gare d'Aguessac
7	Gare d'Aiguebelette-le-Lac
8	Gare d'Aiguebelle
9	Gare d'Aigueblanche
10	Gare d'Aiguillon

3.3. Who are the drivers for TER trains?

We had a hint to look at for typetrain. So we looked up the ERD diagram to see how the tables are connected and what columns they have in common to join them. So we needed to use 3 tables: Person, Driver, and Train. We just compared the person_id and the path_id to see who the drivers are, and afterward, thanks to the hint, added a constraint with typetrain_name.



The screenshot shows a database query editor with a query window and a data output window. The query window contains the following SQL code:

```
1 select person_firstname, person_lastname
2 from person
3 inner join driver
4 on driver.person_id = person.person_id
5 inner join path
6 on driver.person_id = path.person_id
7 inner join train
8 on path.train_id = train.train_id
9 where typetrain_name = 'TER'
```

The data output window shows the results of the query, displaying 10 rows of data. The columns are person_firstname and person_lastname, both of type character varying (256). The data is as follows:

	person_firstname character varying (256)	person_lastname character varying (256)
1	Angelo	SANS
2	Marianne	GOTARDT
3	Andreea Roxana	CHOLLET
4	Baptiste	JEANVOINE
5	Pierre-Alexandre	MONTORFANO CRUZ SANTOS
6	Michel	VILLATTE
7	Sebastien	MALO
8	Jean-Charles	PENG
9	Ishwarya	FRAPSAUCE
10	Xavier	MONTREUIL

3.4. How many sleeping coach do you have in the database?

We checked all relevant tables, including the coach table, but found no information regarding sleeping coaches. This data will need to be added in the future if we want to perform the query.

3.5. Is Yvon Manac'h a driver? A staff member? a Station Chief ? A Mechanics?

Here we needed to check every table one by one and when a query returns the name, that means it is in the table. But this is very long, so we are sure that there is a solution to this. Also we needed your help to use special characters properly in his name.

The first screenshot shows a query for the 'person' table joined with 'revision'. The second screenshot shows a query for the 'person' table joined with 'driver'. The third screenshot shows a query for the 'person' table joined with 'mechanics'. The fourth screenshot shows a query for the 'person' table joined with 'stationchief'. All queries filter for 'Yvon' as the first name and 'MANAC'H' as the last name.

person_firstname	person_lastname
Yvon	MANAC'H

In writing the report, I had an idea: at the end of Lab 1, we used Python and noted it could be powerful for treating data. So, I decided to write a small script with a loop function that allows us to compare names; if a name matches, the program will print it. I apologize for the quality of the screenshots; I made the script using the SSL-VPN portal.

```
[3]: import pycopp2

def main():
    db_config = {
        "dbname": "trains_en",
        "user": "ft_01",
        "password": "Fast",
        "host": "appli-pfe.ec-nantes.fr",
        "port": "5432"
    } #ALL connection details

    target_firstname = "Yvon"
    target_lastname = "MANAC'H"

    tables_to_check = [
        "driver",
        "mechanics",
        "stationchief",
        "revision"
    ] #ALL the tables we want to check

    conn = None
    try:
        conn = pycopp2.connect(**db_config)
        conn.set_client_encoding('UTF8')
        cur = conn.cursor()

        print(f"--- Searching for: {target_firstname} {target_lastname} ---")

        found_any = False #Put False state he will turn True once the name matching
```

```

for table in tables_to_check: # Make a loop to test all tables
    try:
        query = f"""
            SELECT p.person_firstname, p.person_lastname
            FROM person p
            INNER JOIN {table} t ON p.person_id = t.person_id
            WHERE p.person_firstname = %s AND p.person_lastname = %s;
        """

        cur.execute(query, (target_firstname, target_lastname))
        result = cur.fetchone()

        if result: #If target name and name in the tables matching state become true and return the table where the name is.
            print(f"This person is in the table: '{table}'")
            found_any = True
        else:
            pass

    except psycopg2.Error as e:
        print(f"Could not check table '{table}': {e.pgcode} - {e.pgerror.splitlines()[0]}")
        conn.rollback()

if not found_any: #If no matching it returns the sentence below.
    print("--- Result: This person has no known role in the listed tables. ---")

except psycopg2.Error as e:
    print(f"Global connection error: {e}")

finally:
    if conn:
        cur.close()
        conn.close()

```

OUTPUT :

```

--- Searching for: Yvon MANAC'H ---
This person is in the table: 'stationchief'

```

So this person Yvon is well in the table 'stationchief'.

3.6. When did train 85250 arrive in the network?

Here we selected the 'train' table and the column 'train_networkenteringdate' and added the train number constraint, which is 85250.

The screenshot shows a database query interface with a 'Query' tab selected. The query is as follows:

```

1 select train_networkenteringdate from train
2 where train_name = '85250'

```

Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, displaying a table with the following data:

	train_networkenteringdate
1	2005-03-01

The interface also includes a toolbar with icons for various actions and a status bar indicating 'Showing rows: 1 to 1' and 'Page No: 1 of 1'.

3.7. Who are the mechanics who did a revision for train 85250?

Once again, thanks to the ERD diagram, we knew all the links between the tables and made the inner join on the tables to be able to get the data we wanted. Therefore, we needed to select three tables: **mechanics**, **revision**, and **train** to get the needed output, which is the mechanics who did the revision for a specific train.

Query Query History

```
1 select person_firstname, person_lastname from person
2 inner join mechanics
3 on mechanics.person_id = person.person_id
4 inner join revision
5 on mechanics.person_id = revision.person_id
6 inner join train
7 on train.train_id = revision.train_id
8 where train_name = '85250'
```

Data Output Messages Notifications



Showing rows: 1 to 4 Page No: 1 of 1

	person_firstname character varying (256)	person_lastname character varying (256)
1	Susan	ADAM
2	Jonathan	CECCONI
3	Fabien	BINAUD
4	Khaoula	KALLENBACH

3.8. Give the path number for the trains which stop at station Cholet between 6:00 AM and 8:00 AM, on Monday January 2018, 4th.

We passed over an error many times because we used **WHERE** twice (solution: replace with **AND**). And again, with the ERD, we were able to make the right link between the tables:

path, **stop**, **station**.

Query Query History

```
1 select path_number from path
2 inner join stop
3 on path.path_id = stop.path_id
4 inner join station
5 on stop.station_shortcode = station.station_shortcode
6 where station_name like '%Cholet%'
7 and stop_actualltime >= '2018-01-04 06:00:00' and stop_actualltime <= '2018-01-04 08:00:00'
```

Data Output Messages Notifications



Showing rows: 1 to 5 Page No: 1 of 1

	path_number character varying (64)
1	65520
2	70583
3	90558
4	64245
5	82666

3.9. When do the train that stops at Nantes and then stop at Lyon, leave Nantes for January 2018, 4th?

We spent a lot of time on this query because at first, we put two consecutive 'Nantes and Lyon,' but the same train cannot be in two places at the same time. Therefore, we needed to call two copies of the same table (that's why we used nicknames), and thanks to this, we could compare the two stop times to be sure that the train arrives in Nantes before Lyon.

Query	Query History
<pre>1 select s1.stop_actualetime from stop s1 2 inner join stop s2 3 on s1.path_id = s2.path_id 4 inner join station st1 5 on s1.station_shortcode = st1.station_shortcode 6 inner join station st2 7 on s2.station_shortcode = st2.station_shortcode 8 where st1.station_name like '%Nantes%' and st2.station_name like '%Lyon%' 9 and s1.stop_actualetime < s2.stop_actualetime 10 and s1.stop_actualetime >= '2018-01-04 00:00:00' and s2.stop_actualetime <= '2018-01-04 23:59:59';</pre>	
Data Output	Messages Notifications
Showing rows: 1 to 5 Page No: 1 of 1	
stop_actualetime	timestamp without time zone
1	2018-01-04 14:37:00
2	2018-01-04 07:20:00
3	2018-01-04 06:50:00
4	2018-01-04 08:52:00
5	2018-01-04 12:37:00

3.10. According to one of the path you found in 3.9, what are the station stops between Nantes and Lyon ?

We kept the same logic but we chose the path_id for the output where we could choose one.

Query	Query History
<pre>1 select s1.path_id from stop s1 2 inner join stop s2 3 on s1.path_id = s2.path_id 4 inner join station st1 5 on s1.station_shortcode = st1.station_shortcode 6 inner join station st2 7 on s2.station_shortcode = st2.station_shortcode 8 where st1.station_name like '%Nantes%' and st2.station_name like '%Lyon%' 9 and s1.stop_actualetime < s2.stop_actualetime 10 and s1.stop_actualetime >= '2018-01-04 00:00:00' and s2.stop_actualetime <= '2018-01-04 23:59:59';</pre>	
Data Output	Messages Notifications
Showing rows: 1 to 5 Page No: 1 of 1	
path_id	integer
1	500
2	488
3	486
4	491
5	496

We chose the 496 path for this question, so we needed to link the **stop** and **station** tables

using the `station_shortcode`. Thanks to the `path_id`, we could select the train for which we wanted to have its journey details, and we ordered in ascending order to have Nantes first and Lyon last.

Query

Query History

1

2

3

4

5

▼

```
select st.station_name, s.stop_actualetime from stop s
inner join station st
on s.station_shortcode = st.station_shortcode
where s.path_id = 496
order by s.stop_actualetime ASC;
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

	station_name character varying (256) 🔒	stop_actualetime timestamp without time zone 🔒
1	Gare de Nantes	2018-01-04 12:37:00
2	Gare d'Angers Saint-Laud	2018-01-04 13:30:00
3	Gare de Saint-Pierre-des-Corps	2018-01-04 14:28:00
4	Gare de Lyon-Part-Dieu	2018-01-04 17:30:00

3.11. A train which stops at Nantes and that goes to Lyon is broken at the Angers station. We want to call stations between Angers and Lyon to tell the station's chief that the train will be late. Who are the station's chief we will call?

We used tables `path`, `station`, `stop`, `station chief`, and `person` to find the station chiefs at stops on the path we query and verify they meet the requirements laid out by the exercise.

Query

Query History

```

1  select st.station_name, p.person_firstname, p.person_lastname from stop s
2  inner join station st
3  on s.station_shortcode = st.station_shortcode
4  inner join stationchief sc
5  on st.person_id = sc.person_id
6  inner join person p
7  on sc.person_id = p.person_id
8  inner join stop s_angers
9  on s.path_id = s_angers.path_id
10 inner join station st_angers
11 on s_angers.station_shortcode = st_angers.station_shortcode
12 inner join stop s_lyon
13 on s.path_id = s_lyon.path_id
14 inner join station st_lyon
15 on s_lyon.station_shortcode = st_lyon.station_shortcode
16 where s.path_id = 500
17 and st_angers.station_name like '%Angers%'
18 and st_lyon.station_name like '%Lyon%'
19 and s.stop_actualetime > s_angers.stop_actualetime
20 and s.stop_actualetime <= s_lyon.stop_actualetime;

```

Data Output

Messages

Notifications

+

▼

📁

▼

🗑️

📄

⌂

SQL

	station_name character varying (256)	person_firstname character varying (256)	person_lastname character varying (256)
1	Gare de Lyon-Part-Dieu	Weijun	ALOUANE
2	Gare de Saint-Pierre-des-Corps	Maxime	CONTE

4.1. How many drivers do you have in the database?

We used the **COUNT** function to ask SQL to perform the calculation 'count,' and it returned all the drivers.

Query Query History

1 select count(person_id) from driver

Data Output Messages Notifications

≡+

▼

▼

SQL

	count bigint
1	101

4.2. On how many path did each driver drive a train ?

we start by selecting the **driver** table and linking them to the **person** table to retrieve their actual names. then use a **left join** with the **path** table, which connects drivers to the specific trips they have made using their **person_id**. group the results by each individual driver and count the number of path entries.

Query

Query History

1

2

3

4

5

6

7

8

▼

```
select p.person_firstname, p.person_lastname,
count(pa.path_id) as number_of_paths
from driver d
inner join person p
on d.person_id = p.person_id
left join path pa
on d.person_id = pa.person_id
group by p.person_id, p.person_firstname, p.person_lastname;
```

Data Output

Messages

Notifications

≡

▼

▼

SQL

	person_firstname character varying (256)	person_lastname character varying (256)	number_of_paths bigint
1	Flavio	RUBLON	1156
2	Thomas	ANANT	1156
3	Rebiha	MULLER	1156
4	Pierre	ABRAHAM	1156
5	Yoann	GUITTON	1157
6	Baptiste	JEANVOINE	1156
7	Julien	LADACI	1157
8	Christophe	DE CASANOVE	1156
9	Mathilde	GUILLOPE	1157
10	Cedric	DENIS	1157

4.3. What is the maximum path time, that means the max of the time difference between 2 station's stops on the same path ?

The initial query returned a null result due to `stop_actualetime` missing, likely representing unscheduled or future stops. Sorting by descending order places these null values at the top, hiding the real data. The query was adjusted to filter out null and order the results with `NULLS LAST`; the longest valid journey duration was correctly identified and returned.

Removing LIMIT 1 would result in a very long table displaying the duration for every single recorded trip.

The screenshot shows a SQL query editor with the following query:

```
1 select max(s.stop_actualetime) - min(s.stop_actualetime) as max_path_duration
2 from stop s
3 where s.stop_actualetime is not null
4 group by s.path_id
5 order by max_path_duration desc nulls last
6 limit 1
```

Below the query editor, the 'Data Output' tab is active, showing a table with one row and one column:

	max_path_duration interval
1	07:00:00

4.4 What is the max delay time on 2018-11-03 and at which station stop ?

calculate the delay for every stop recorded on **November 3, 2018**, by subtracting the estimated time from the actual time. We then join the station table to retrieve the name of the station. We order the results in descending order (largest delay first) and limit the result to

The screenshot shows a SQL query editor with the following query:

```
1 select st.station_name, (s.stop_actualetime - s.stop_estimatedtime) as max_delay
2 from stop s
3 inner join station st
4 on s.station_shortcode = st.station_shortcode
5 where s.stop_actualetime >= '2018-11-03 00:00:00' and s.stop_actualetime <= '2018-11-03 23:59:59'
6 order by max_delay desc
7 limit 1
```

Below the query editor, the 'Data Output' tab is active, showing a table with two columns and one row:

	station_name character varying (256)	max_delay interval
1	Gare de Nantes	00:29:00

4.5. What is the mean of the cleaning time of the coaches that took part of a path on 2018-11-03?

We were unable to execute the query related to cleaning duration because the relevant tables lack a column or field that specifies the duration for the cleaning activities. Therefore, we do not have the necessary data to calculate or retrieve this information.

4.6. What are the driving time of each driver, on each path, in October 2018?

Explanation

1. **with path_duration as (...)**: This defines the temporary table.
 - It filters the **stop** table for records in **October 2018**.
 - It groups by **path_id** and calculates the difference between the arrival (**max**) and departure (**min**) times.
2. **join path pa**: We link the calculated times back to the path information.
3. **join driver d**: The **path** table contains the **person_id** of the driver assigned to that path.
4. **join person p**: Finally, we retrieve the human-readable names.

Query

Query History

```
1 with path_duration as (  
2     select s.path_id,  
3     max(s.stop_actualetime) - min(s.stop_actualetime) as driving_time  
4     from stop s  
5     where s.stop_actualetime >= '2018-10-01 00:00:00'  
6     and s.stop_actualetime <= '2018-10-31 23:59:59'  
7     group by s.path_id  
8 )  
9 select p.person_firstname, p.person_lastname, pa.path_id, pd.driving_time  
10 from path_duration pd  
11 join path pa  
12 on pd.path_id = pa.path_id  
13 inner join driver d  
14 on pa.person_id = d.person_id  
15 inner join person p  
16 on d.person_id = p.person_id;
```

Data Output

Messages

Notifications

SQL

	person_firstname character varying (256)	person_lastname character varying (256)	path_id integer	driving_time interval
1	Tristan	BAYART	44642	01:44:00
2	Patrice	FAIVRE	43737	01:26:00
3	Eric	MOUREAU	47289	01:02:00
4	T	GLATRE	45916	01:05:00
5	Sarah	SIDIBE	47492	00:29:00
6	Yun	PEROCHEAU	45408	00:30:00
7	Victor-Luis	PETIT	45178	05:30:00
8	Philippe	RIGOUREAU	45014	02:11:00
9	Renaud	BARBA	43885	03:24:00
10	Segolene	ROYAL	43792	01:15:00

Conclusion

This lab successfully utilized the ERD and `INNER JOIN` to navigate the complex train database and construct required queries. Challenges included schema/database mismatches (e.g., `staff` vs. `agent`), filtering null data for time calculations, and using table aliases for sequential stop queries. The exercise also highlighted dataset limitations, such as missing "sleeping coaches" and "cleaning duration" data, preventing certain analytical queries. In summary, the lab emphasized the practical application of SQL and the critical need for pre-execution database structure mapping.

Date : 18/12/2025

Thank for reading

Adriel